

CA Agent

Complete Implementation Guide

Extension-by-extension code, test commands, and deployment — from existing prototype to live cloud app

Hardware: i5-12th gen · 16 GB RAM · 4 GB VRAM

How to use this guide. Work through each step in order. Every step has: the exact file to create or edit, the complete code to write, and a terminal command to verify it works before moving on. Do not skip the test for each step — each one is the foundation for the next.

Contents

1	Before You Start — Repository Setup	3
1.0.1	Verify your current project runs	3
2	Step 1 — Rolling Conversational Memory	3
2.0.1	1.1 Create the memory module	3
2.0.2	1.2 Update api.py	4
2.0.3	1.3 Update ca_agent.py	5
2.0.4	1.4 Update ChatContext.tsx	6
3	Step 2 — Multi-Document Cross-Analysis	7
3.0.1	2.1 Add merge_doc_data to ca_agent.py	7
3.0.2	2.2 Use merge_doc_data in /ask	8
3.0.3	2.3 Create Dashboard.tsx	8
3.0.4	2.4 Add /dashboard route to App.tsx	10
4	Step 3 — Anomaly Detection on Transactions	10
4.0.1	3.1 Create anomaly.py	10
4.0.2	3.2 Wire into api.py /upload endpoint	11
4.0.3	3.3 Add transactions to Gemini Vision prompt	12
5	Step 4 — RAGAS Evaluation Harness	13
5.0.1	4.1 Install RAGAS	13
5.0.2	4.2 Generate the golden dataset with Gemini	13
5.0.3	4.3 Create run_eval.py	14
5.0.4	4.4 Expose retrieval chunks from /ask	15
6	Step 5 — Fine-Tuned Domain Embeddings	15
6.0.1	5.1 Install sentence-transformers	15
6.0.2	5.2 Create fine_tune_embeddings.py	15
6.0.3	5.3 Regenerate the .joblib files	16
6.0.4	5.4 Update search.py to remove Ollama	17
7	Step 6 — GraphRAG on Income Tax Act	18
7.0.1	6.1 Install networkx	18
7.0.2	6.2 Create build_tax_graph.py	18
7.0.3	6.3 Create graph_retrieve.py	19

7.0.4	6.4 Blend graph results into search.py	20
8	Step 7 — Hallucination Detection and Source Badges	20
8.0.1	7.1 Create hallucination_guard.py	21
8.0.2	7.2 Call it in api.py	22
8.0.3	7.3 Render the badge in ChatMessage.tsx	22
9	Step 8 — Agentic ITR Filing Assistant	23
9.0.1	8.1 Create itr_agent.py	23
9.0.2	8.2 Add /itr endpoint in api.py	25
9.0.3	8.3 Create ITRWizard.tsx	26
10	Step 9 — Compliance Calendar and Deadline Alerts	27
10.0.1	9.1 Create compliance_calendar.py	27
10.0.2	9.2 Add /calendar endpoint	29
10.0.3	9.3 Create CalendarView.tsx with push notifications	29
11	Step 10 — Voice Interface	31
11.0.1	10.1 Create useSpeech.ts hook	31
11.0.2	10.2 Add mic button to ChatInput.tsx	32
11.0.3	10.3 Add read-aloud button to ChatMessage.tsx	32
12	Step 11 — Containerise the Backend	33
12.0.1	11.1 Create requirements.txt	33
12.0.2	11.2 Create Dockerfile	33
12.0.3	11.3 Create docker-compose.yml	34
12.0.4	11.4 Create .dockerignore	34
13	Step 12 — Cloud Deployment	35
13.0.1	12.1 Deploy backend to Render.com	35
13.0.2	12.2 Deploy frontend to Vercel	36
13.0.3	12.3 Fix CORS for production	36
13.0.4	12.4 Demo-day fallback: Cloudflare Tunnel	37
14	Final Checklist	38

1. Before You Start — Repository Setup

- **Verify your current project runs**

```

1 # terminal 1 -- start Ollama (still needed at this stage)
2 ollama serve
3
4 # terminal 2 -- start backend
5 cd your-project/
6 uvicorn api:app --reload --host 127.0.0.1 --port 8000
7
8 # terminal 3 -- start frontend
9 npm run dev

```

Open <http://localhost:5173>, upload a bank statement, ask a question. If this works, you have a confirmed baseline. Now create a new Git branch:

```

1 git checkout -b fyp-extensions
2 git add -A && git commit -m "baseline: confirmed working prototype"

```

Commit after every step

Each step ends with a `git commit`. If something breaks, you can always roll back with `git checkout step-N` without losing earlier work.

2. Step 1 — Rolling Conversational Memory

Goal

Replace the hardcoded 3-turn window in `api.py` with a rolling summary so the agent retains key financial facts across long sessions.

Files touched

Edit: `api.py` Edit: `src/contexts/ChatContext.tsx`

- **1.1 Create the memory module**

Create a new file `memory.py` in your project root:

```

1 # memory.py -- rolling conversation summary
2 import google.generativeai as genai
3
4 SUMMARY_PROMPT = """You are a CA assistant memory compressor.
5 Given the following conversation turns, extract and summarise ONLY
6 the financially relevant facts in bullet points. Focus on:
7 - Uploaded document details (person, income, TDS, period)
8 - Tax figures mentioned (deductions, liabilities, advance tax)
9 - Decisions or conclusions the user and assistant reached
10 - Any pending questions or missing documents
11
12 Keep the summary under 200 words. Return plain text bullets only.

```

```

13
14 Conversation to compress:
15 {conversation}
16 """
17
18 def summarise_turns(turns: list[dict], model_name: str = "gemini-1.5-flash")
19     -> str:
20     """Compress older turns into a financial facts summary."""
21     if not turns:
22         return ""
23     model = genai.GenerativeModel(model_name)
24     formatted = "\n".join(
25         f"{t['role'].upper()}: {t['content']}" for t in turns
26     )
27     try:
28         response =
29             model.generate_content(SUMMARY_PROMPT.format(conversation=formatted))
30         return response.text.strip()
31     except Exception as e:
32         # Graceful fallback: return empty string so /ask still works
33         print(f"[memory] summarisation failed: {e}")
34         return ""
35
36 def build_context(history: list[dict], keep_recent: int = 6) -> dict:
37     """
38     Split history into:
39     - summary: compressed older turns (string)
40     - recent: last 'keep_recent' turns verbatim (list)
41
42     Args:
43     history:      full conversation as [{role, content}, ...]
44     keep_recent:  how many recent turns to keep verbatim (default 6 = 3
45                     pairs)
46
47     Returns:
48     {"summary": str, "recent": list[dict]}
49     """
50     if len(history) <= keep_recent:
51         return {"summary": "", "recent": history}
52
53     old_turns = history[:-keep_recent]
54     recent_turns = history[-keep_recent:]
55     summary = summarise_turns(old_turns)
56     return {"summary": summary, "recent": recent_turns}

```

• 1.2 Update api.py

Find the section in `api.py` where chat history is built and replace it:

```

1 # api.py -- at the top, add this import
2 from memory import build_context
3
4 # In your AskRequest model, add the summary field:
5 class AskRequest(BaseModel):
6     question: str
7     history: list[dict] = []
8     doc_data: list[dict] = []
9     plain_file_texts: list[str] = []
10     # NEW field -- frontend sends this back on every request
11     conversation_summary: str = ""
12
13 # In your /ask endpoint, replace the history-slicing block:

```

```

14 @app.post("/ask")
15 async def ask(req: AskRequest):
16     # Build rolling memory context
17     memory_ctx = build_context(req.history, keep_recent=6)
18     summary     = req.conversation_summary or memory_ctx["summary"]
19     recent      = memory_ctx["recent"]
20
21     # Return the new summary so frontend can persist it
22     new_summary = memory_ctx["summary"] if not req.conversation_summary else
        summary
23
24     # Pass summary + recent turns to ca_agent
25     answer = process_query(
26         question      = req.question,
27         history        = recent,
28         memory_summary = summary,          # NEW parameter
29         doc_data       = req.doc_data,
30         plain_file_texts= req.plain_file_texts,
31     )
32     return {
33         "answer":      answer,
34         "summary":     new_summary,      # NEW -- frontend stores this
35         "retrieval":   [],
36     }

```

• 1.3 Update ca_agent.py

Find process_query and add the memory_summary parameter:

```

1  # ca_agent.py -- update process_query signature and prompt assembly
2
3  def process_query(
4      question:      str,
5      history:        list[dict],
6      memory_summary: str = "",          # NEW
7      doc_data:       list[dict] = [],
8      plain_file_texts: list[str] = [],
9  ) -> str:
10
11     # Build the system prompt section for memory
12     memory_block = ""
13     if memory_summary:
14         memory_block = f"""
15 CONVERSATION MEMORY (key facts from earlier in this session):
16 {memory_summary}
17
18 ---
19 """
20     # Prepend memory_block to your existing prompt construction
21     # Find where you build 'prompt' and add memory_block before history
22     # Example:
23     system = f"""You are an expert CA assistant specialising in Indian
24         taxation.
25 {memory_block}
26 Recent conversation:
27 {format_history(history)}
28
29 Document context:
30 {format_doc_context(doc_data, plain_file_texts)}
31
32 Question: {question}
33 Answer in clear, structured English. Cite relevant sections of the Income Tax

```

```

    Act or GST law where applicable.
33 """
34     model      = genai.GenerativeModel("gemini-1.5-pro")
35     response = model.generate_content(system)
36     return response.text

```

● 1.4 Update ChatContext.tsx

```

1 // src/contexts/ChatContext.tsx
2 // 1. Add summary field to Conversation type
3 interface Conversation {
4     id:      string;
5     title:   string;
6     messages: Message[];
7     summary: string; // NEW -- rolling memory summary
8     // ... existing fields
9 }
10
11 // 2. In sendMessage(), include summary in the request body:
12 const response = await fetch(`${API_URL}/ask`, {
13     method: "POST",
14     headers: { "Content-Type": "application/json" },
15     body: JSON.stringify({
16         question:      userMessage,
17         history:        conversationHistory,
18         doc_data:       activeConversation.docData,
19         plain_file_texts: activeConversation.plainFileTexts,
20         conversation_summary: activeConversation.summary, // NEW
21     }),
22 });
23
24 const data = await response.json();
25
26 // 3. Store the returned summary back onto the conversation:
27 updateConversation(activeConversation.id, {
28     messages: [...activeConversation.messages, assistantMessage],
29     summary:  data.summary ?? activeConversation.summary, // NEW
30 });

```

How to test

```

1 # Restart backend, open app, upload a statement, send 5+ messages.
2 # On message 7, ask: "What was the income on the document I uploaded?"
3 # The agent should answer correctly even though the upload was > 3 turns
  ago.
4 # Check terminal -- you should see no errors from memory.py.

```

```

1 git add memory.py api.py ca_agent.py src/contexts/ChatContext.tsx
2 git commit -m "step-1: rolling conversational memory"

```

3. Step 2 — Multi-Document Cross-Analysis

Goal

Aggregate extracted fields across all uploaded documents so the agent can answer questions about quarterly totals, trends, and cross-statement anomalies.

Files touched

Edit: ca_agent.py Edit: api.py New: src/components/Dashboard.tsx Edit: src/App.tsx

• 2.1 Add merge_doc_data to ca_agent.py

```
1  # ca_agent.py -- add below existing imports
2
3  from typing import Optional
4  import re
5
6  def merge_doc_data(docs: list[dict]) -> dict:
7      """
8      Aggregate multiple doc_data payloads from different uploads
9      into a single unified financial summary.
10
11      Each doc_data dict is expected to have keys produced by
12      file_handler.py: total_credits, total_debits, statement_period,
13      person_entity, estimated_annual_income, tds_deducted, etc.
14      """
15      if not docs:
16          return {}
17      if len(docs) == 1:
18          return docs[0]
19
20      def safe_float(val) -> float:
21          """Convert string rupee amounts like '45,230.00' to float."""
22          if val is None:
23              return 0.0
24          return float(re.sub(r"[^\d.]", "", str(val)) or 0)
25
26      total_credits = sum(safe_float(d.get("total_credits")) for d in docs)
27      total_debits = sum(safe_float(d.get("total_debits")) for d in docs)
28      total_tds = sum(safe_float(d.get("tds_deducted")) for d in docs)
29      avg_income = total_credits / len(docs)
30
31      periods = [d.get("statement_period", "unknown") for d in docs]
32
33      monthly_breakdown = []
34      for d in docs:
35          monthly_breakdown.append({
36              "period": d.get("statement_period", "?"),
37              "credits": safe_float(d.get("total_credits")),
38              "debits": safe_float(d.get("total_debits")),
39              "tds": safe_float(d.get("tds_deducted")),
40          })
41
42      # Estimate advance tax liability (simplified: 30% of net income)
43      net_annual = (total_credits - total_debits) * (12 / max(len(docs), 1))
44      est_tax = max(0, net_annual - 250000) * 0.05 # simplified slab
45
46      return {
47          "person_entity": docs[0].get("person_entity", ""),
48          "documents_merged": len(docs),
49          "periods_covered": periods,
50          "total_credits": round(total_credits, 2),
51          "total_debits": round(total_debits, 2),
52          "net_cashflow": round(total_credits - total_debits, 2),
53          "total_tds_deducted": round(total_tds, 2),
54          "average_monthly_income": round(avg_income, 2),
55          "estimated_annual_net": round(net_annual, 2),
```

```

56     "estimated_basic_tax":    round(est_tax,        2),
57     "monthly_breakdown":     monthly_breakdown,
58 }

```

● 2.2 Use merge_doc_data in /ask

```

1  # api.py -- in /ask endpoint, before calling process_query:
2
3  merged = merge_doc_data(req.doc_data) if len(req.doc_data) > 1 else (
4      req.doc_data[0] if req.doc_data else {}
5  )
6
7  # Pass merged instead of raw doc_data list:
8  answer = process_query(
9      question      = req.question,
10     history       = recent,
11     memory_summary = summary,
12     doc_data      = [merged],          # single merged dict
13     plain_file_texts = req.plain_file_texts,
14 )
15 return {
16     "answer":      answer,
17     "summary":     new_summary,
18     "merged_data": merged,             # send to frontend for dashboard
19 }

```

● 2.3 Create Dashboard.tsx

```

1  // src/components/Dashboard.tsx
2  import { useChat } from "../contexts/ChatContext";
3  import {
4      BarChart, Bar, XAxis, YAxis, Tooltip, Legend,
5      PieChart, Pie, Cell, ResponsiveContainer
6  } from "recharts";
7
8  const COLORS = ["#0F6E56", "#534AB7", "#993C1D", "#854F0B"];
9
10 export default function Dashboard() {
11     const { activeConversation } = useChat();
12     const docs = activeConversation?.uploadedDocs ?? [];
13
14     if (docs.length === 0) {
15         return (
16             <div className="flex items-center justify-center h-full text-gray-400">
17                 Upload bank statements to see your financial dashboard.
18             </div>
19         );
20     }
21
22     // Build chart data from uploaded doc_data
23     const barData = docs.map((doc) => ({
24         name:      doc.statement_period ?? "Unknown",
25         Credits:   parseFloat(doc.total_credits ?? 0),
26         Debits:    parseFloat(doc.total_debits ?? 0),
27         TDS:       parseFloat(doc.tds_deducted ?? 0),
28     }));
29
30     const totalCredits = barData.reduce((s, d) => s + d.Credits, 0);
31     const totalDebits  = barData.reduce((s, d) => s + d.Debits, 0);
32     const pieData = [
33         { name: "Net Income", value: Math.max(0, totalCredits - totalDebits) },

```

```

34     { name: "Total Debits", value: totalDebits },
35 ];
36
37 return (
38   <div className="p-6 space-y-8 overflow-y-auto h-full">
39     <h2 className="text-xl font-semibold text-gray-800">
40       Financial Overview ({docs.length} statement{docs.length > 1 ? "s" : ""})
41     </h2>
42
43     { /* Summary cards */ }
44     <div className="grid grid-cols-3 gap-4">
45       {[
46         { label: "Total Credits", value: `Rs.
47           ${totalCredits.toLocaleString("en-IN")}` },
48         { label: "Total Debits", value: `Rs.
49           ${totalDebits.toLocaleString("en-IN")}` },
50         { label: "Net Cash Flow", value: `Rs. ${((totalCredits -
51           totalDebits).toLocaleString("en-IN"))}` },
52       ]}.map((c) => (
53         <div key={c.label} className="bg-white rounded-xl border
54           border-gray-200 p-4">
55           <p className="text-sm text-gray-500">{c.label}</p>
56           <p className="text-lg font-bold text-gray-900 mt-1">{c.value}</p>
57         </div>
58       )))
59     </div>
60
61     { /* Bar chart: credits vs debits per month */ }
62     <div className="bg-white rounded-xl border border-gray-200 p-4">
63       <p className="text-sm font-medium text-gray-700 mb-3">Monthly Credits vs
64         Debits</p>
65       <ResponsiveContainer width="100%" height={240}>
66         <BarChart data={barData}>
67           <XAxis dataKey="name" tick={{ fontSize: 11 }} />
68           <YAxis tick={{ fontSize: 11 }} />
69           <Tooltip formatter={(v) => `Rs.
70             ${Number(v).toLocaleString("en-IN")}` } />
71           <Legend />
72           <Bar dataKey="Credits" fill="#0F6E56" radius={[4,4,0,0]} />
73           <Bar dataKey="Debits" fill="#993C1D" radius={[4,4,0,0]} />
74         </BarChart>
75       </ResponsiveContainer>
76     </div>
77
78     { /* Pie chart: income split */ }
79     <div className="bg-white rounded-xl border border-gray-200 p-4">
80       <p className="text-sm font-medium text-gray-700 mb-3">Income
81         Distribution</p>
82       <ResponsiveContainer width="100%" height={200}>
83         <PieChart>
84           <Pie data={pieData} cx="50%" cy="50%" outerRadius={80}
85             dataKey="value" label={({ name, percent }) =>
86               `${name} ${(percent * 100).toFixed(0)}%`>
87             {pieData.map((_, i) => (
88               <Cell key={i} fill={COLORS[i % COLORS.length]} />
89             ))}
90           </Pie>
91           <Tooltip formatter={(v) => `Rs.
92             ${Number(v).toLocaleString("en-IN")}` } />
93         </PieChart>
94       </ResponsiveContainer>
95     </div>
96   </div>

```

```

89     );
90 }

```

• 2.4 Add /dashboard route to App.tsx

```

1 // src/App.tsx -- add import and route
2 import Dashboard from "../components/Dashboard";
3
4 // Inside your router:
5 <Route path="/dashboard" element={<Dashboard />} />
6 // Also add a nav link in your header/sidebar pointing to /dashboard

```

How to test

Upload 2 or 3 bank statements in one conversation. Navigate to /dashboard. You should see the bar chart and summary cards populated with aggregated data. Check that totals across statements are summed correctly.

```

1 git add ca_agent.py api.py src/components/Dashboard.tsx src/App.tsx
2 git commit -m "step-2: multi-document cross-analysis and dashboard"

```

4. Step 3 — Anomaly Detection on Transactions

Goal

Flag statistically unusual transactions in uploaded statements using IsolationForest. Append plain-English explanations to the upload response. Runs entirely on CPU.

Files touched

New: anomaly.py Edit: api.py

• 3.1 Create anomaly.py

```

1 # anomaly.py
2 import re
3 import numpy as np
4 from sklearn.ensemble import IsolationForest
5
6 def _to_float(val) -> float:
7     if val is None:
8         return 0.0
9     return float(re.sub(r"^\d.", "", str(val)) or 0)
10
11 def detect_anomalies(transactions: list[dict]) -> list[dict]:
12     """
13     Find anomalous transactions using IsolationForest.
14
15     Args:
16         transactions: list of dicts with at least {"date": str,
17             "description": str, "amount": str|float,
18             "type": "credit"|"debit"}
19
20     Returns:
21         list of anomalous transaction dicts, each with an added

```

```

21     "anomaly_reason" key explaining why it was flagged.
22     """
23     if len(transactions) < 5:
24         # Too few transactions to detect anomalies reliably
25         return []
26
27     amounts = np.array([_to_float(t.get("amount")) for t in transactions])
28     mean     = amounts.mean()
29     std      = amounts.std()
30
31     # IsolationForest expects 2-D input
32     X        = amounts.reshape(-1, 1)
33     clf      = IsolationForest(contamination=0.07, random_state=42)
34     labels   = clf.fit_predict(X)    # -1 = anomaly
35
36     flagged = []
37     for txn, label, amt in zip(transactions, labels, amounts):
38         if label == -1:
39             ratio = amt / mean if mean > 0 else 1.0
40             direction = "credit" if txn.get("type", "").lower() == "credit"
41                 else "debit"
42             reason = (
43                 f"This {direction} of Rs. {amt:,.0f} is "
44                 f"{ratio:.1f}x your average transaction amount "
45                 f"(avg Rs. {mean:,.0f}). "
46             )
47             if ratio > 5:
48                 reason += "Large one-time amounts may affect advance tax"
49                     liability."
50             elif std > 0 and abs(amt - mean) > 2.5 * std:
51                 reason += "Statistically unusual -- verify this transaction."
52             else:
53                 reason += "Consider discussing this with your CA."
54
55             flagged.append({**txn, "anomaly_reason": reason})
56
57     return flagged
58
59 def anomaly_summary_text(flagged: list[dict]) -> str:
60     """Convert flagged list to a human-readable summary for the chat."""
61     if not flagged:
62         return ""
63     lines = ["\n**Flagged transactions requiring attention:**\n"]
64     for txn in flagged:
65         lines.append(
66             f"- {txn.get('date', 'Unknown date')}: "
67             f"{txn.get('description', 'Transaction')} "
68             f"(Rs. {_to_float(txn.get('amount')):.0f}) -- "
69             f"{txn.get('anomaly_reason', '')}"
70         )
71     return "\n".join(lines)

```

● 3.2 Wire into api.py /upload endpoint

```

1  # api.py -- add import at top
2  from anomaly import detect_anomalies, anomaly_summary_text
3
4  # In your /upload endpoint, after Gemini Vision returns doc_data:
5  # Assume doc_data may have a "transactions" key with a list of txns.
6  # If your Gemini prompt does not extract individual transactions yet,
7  # add this to your file_handler.py prompt (see note below).
8

```

```

9 transactions = doc_data.get("transactions", [])
10 flagged      = detect_anomalies(transactions)
11 anomaly_text = anomaly_summary_text(flagged)
12
13 # Append anomaly text to summary_message:
14 existing_summary = doc_data.get("summary_message", "")
15 doc_data["summary_message"] = existing_summary + anomaly_text
16 doc_data["flagged_transactions"] = flagged
17
18 return {
19     "doc_item":      doc_item,
20     "doc_data":      doc_data,
21     "summary_message": doc_data["summary_message"],
22     "plain_text_append": plain_text,
23 }

```

• 3.3 Add transactions to Gemini Vision prompt

In `file_handler.py`, extend the JSON output schema in your prompt:

```

1  # file_handler.py -- extend the strict JSON schema in your Gemini prompt
2  # Find your EXTRACTION_PROMPT string and add the transactions field:
3
4  EXTRACTION_PROMPT = """
5  Analyse this bank statement image/PDF and return ONLY valid JSON.
6  No markdown, no explanation, only the JSON object.
7
8  Required JSON structure:
9  {
10     "person_entity":      "full name or company name",
11     "account_number":     "last 4 digits only",
12     "statement_period":   "Mon YYYY - Mon YYYY",
13     "opening_balance":    "0.00",
14     "closing_balance":    "0.00",
15     "total_credits":      "0.00",
16     "total_debits":       "0.00",
17     "tds_deducted":       "0.00",
18     "estimated_annual_income": "0.00",
19     "transactions": [
20         {
21             "date":        "DD Mon YYYY",
22             "description": "short description",
23             "amount":      "0.00",
24             "type":        "credit or debit"
25         }
26     ]
27 }
28
29 Extract ALL visible transactions. If a field is not present, use null.
30 """

```

How to test

Upload a bank statement with at least 10 transactions. The chat response should now include a “Flagged transactions” section for any unusual amounts. Verify in the terminal that `anomaly.py` runs without errors.

```

1  pip install scikit-learn --break-system-packages # if not already installed
2  git add anomaly.py api.py file_handler.py
3  git commit -m "step-3: anomaly detection on extracted transactions"

```

5. Step 4 — RAGAS Evaluation Harness

Goal

Build a golden QA dataset and a repeatable evaluation script so every retrieval improvement can be measured with real numbers.

Files touched

New: eval_dataset.json New: run_eval.py Edit: metrics.py

• 4.1 Install RAGAS

```
1 pip install ragas datasets --break-system-packages
```

• 4.2 Generate the golden dataset with Gemini

```
1 # generate_eval_dataset.py -- run once to build eval_dataset.json
2 # Place your ICAI PDF text in a list called 'corpus_chunks'.
3
4 import json, google.generativeai as genai
5
6 genai.configure(api_key="YOUR_GEMINI_API_KEY")
7 model = genai.GenerativeModel("gemini-1.5-flash")
8
9 PROMPT = """Given this excerpt from ICAI study material, generate
10 3 question-answer pairs that a CA student or client might ask.
11 Return ONLY a JSON array with objects having keys:
12 "question", "ground_truth", "source_chunk"
13 No markdown fences.
14
15 Excerpt:
16 {chunk}
17 """
18
19 corpus_chunks = [
20     # Paste 30-40 representative ICAI text chunks here
21     # Example:
22     "Section 44AD: An eligible assessee (individual, HUF, or "
23     "partnership firm, not LLP) engaged in any eligible business "
24     "can declare income at 8% of gross turnover or gross receipts "
25     "for the previous year on or before the due date for filing return.",
26     # ... add more chunks from your .joblib corpus texts
27 ]
28
29 dataset = []
30 for chunk in corpus_chunks:
31     try:
32         resp = model.generate_content(PROMPT.format(chunk=chunk))
33         pairs = json.loads(resp.text)
34         dataset.extend(pairs)
35         print(f"Generated {len(pairs)} pairs. Total: {len(dataset)}")
36     except Exception as e:
37         print(f"Skipped chunk: {e}")
38
39 with open("eval_dataset.json", "w") as f:
40     json.dump(dataset, f, indent=2)
```

```
41 print(f"Saved {len(dataset)} Q&A pairs to eval_dataset.json")
```

• 4.3 Create run_eval.py

```
1  # run_eval.py -- run before and after each retrieval improvement
2  import json, requests
3  from datasets import Dataset
4  from ragas import evaluate
5  from ragas.metrics import (
6      faithfulness,
7      answer_relevancy,
8      context_recall,
9      context_precision,
10 )
11
12 API_URL = "http://127.0.0.1:8000/ask"
13
14 def get_answer_and_contexts(question: str) -> dict:
15     """Call your /ask endpoint and return answer + retrieved chunks."""
16     resp = requests.post(API_URL, json={
17         "question": question,
18         "history": [],
19         "doc_data": [],
20         "plain_file_texts": [],
21         "conversation_summary": "",
22     })
23     data = resp.json()
24     return {
25         "answer": data.get("answer", ""),
26         "contexts": data.get("retrieval_chunks", []), # add this to /ask
27             response
28     }
29
30 def run_evaluation(dataset_path: str = "eval_dataset.json"):
31     with open(dataset_path) as f:
32         raw = json.load(f)
33
34     print(f"Running evaluation on {len(raw)} questions...")
35
36     rows = []
37     for item in raw:
38         q = item["question"]
39         gt = item["ground_truth"]
40         out = get_answer_and_contexts(q)
41         rows.append({
42             "question": q,
43             "answer": out["answer"],
44             "contexts": out["contexts"] if out["contexts"] else
45                 [item["source_chunk"]],
46             "ground_truth": gt,
47         })
48
49     dataset = Dataset.from_list(rows)
50     result = evaluate(
51         dataset,
52         metrics=[faithfulness, answer_relevancy, context_recall,
53                 context_precision],
54     )
55
56     print("\n=== RAGAS Evaluation Results ===")
57     print(result)
58     result.to_pandas().to_csv("eval_results.csv", index=False)
59     print("Results saved to eval_results.csv")
```

```

56
57 if __name__ == "__main__":
58     run_evaluation()

```

• 4.4 Expose retrieval chunks from /ask

```

1 # api.py -- in /ask endpoint, include chunk texts in the response:
2 # After search_rag() returns, add chunk texts to response:
3
4 return {
5     "answer":          answer,
6     "summary":         new_summary,
7     "retrieval_chunks": [chunk["text"] for chunk in top_chunks], # NEW
8     "retrieval":       top_chunks,
9 }

```

How to test

```

1 # Make sure backend is running, then:
2 python run_eval.py
3 # You should see 4 metric scores printed. Save these numbers --
4 # they are your BASELINE. You will re-run this after Step 6
5 # (fine-tuned embeddings) to show improvement.

```

```

1 git add eval_dataset.json run_eval.py generate_eval_dataset.py
2 git commit -m "step-4: RAGAS evaluation harness with golden dataset"

```

6. Step 5 — Fine-Tuned Domain Embeddings

Goal

Fine-tune all-MiniLM-L6-v2 on ICAI Q&A pairs so the embedding model understands Indian tax terminology. Replace the Ollama/bge-m3 dependency. Regenerate all three `.joblib` files.

Files touched

New: `fine_tune_embeddings.py` New: `regenerate_joblib.py` Edit: `search.py`

• 5.1 Install sentence-transformers

```

1 pip install sentence-transformers torch --break-system-packages

```

• 5.2 Create `fine_tune_embeddings.py`

```

1 # fine_tune_embeddings.py
2 import json
3 from sentence_transformers import (
4     SentenceTransformer, InputExample, losses, evaluation
5 )
6 from torch.utils.data import DataLoader
7
8 # Load your golden dataset from Step 4 as training pairs

```

```

9  with open("eval_dataset.json") as f:
10     raw = json.load(f)
11
12     # Build positive pairs: (question, source_chunk)
13     # These teach the model that this question belongs near this text
14     train_examples = [
15         InputExample(texts=[item["question"], item["source_chunk"]])
16         for item in raw
17         if item.get("source_chunk")
18     ]
19
20     # For better training, also add (ground_truth, source_chunk) pairs
21     train_examples += [
22         InputExample(texts=[item["ground_truth"], item["source_chunk"]])
23         for item in raw
24         if item.get("source_chunk")
25     ]
26
27     print(f"Training on {len(train_examples)} pairs...")
28
29     # Load base model -- 80 MB, fits in 4 GB VRAM easily
30     model = SentenceTransformer("all-MiniLM-L6-v2")
31     loader = DataLoader(train_examples, shuffle=True, batch_size=16)
32
33     # MultipleNegativesRankingLoss: treats all other items in the batch
34     # as negatives -- no need to manually curate negative examples
35     loss = losses.MultipleNegativesRankingLoss(model)
36
37     # Train
38     model.fit(
39         train_objectives=[(loader, loss)],
40         epochs=3,
41         warmup_steps=max(10, len(train_examples) // 10),
42         show_progress_bar=True,
43         output_path="models/icai-embeddings",
44     )
45
46     print("Fine-tuning complete. Model saved to models/icai-embeddings/")
47     print("Run regenerate_joblib.py next to rebuild the .joblib files.")

```

• 5.3 Regenerate the .joblib files

```

1  # regenerate_joblib.py
2  # This re-embeds all your existing corpus chunks with the new model
3  # and saves them back as .joblib files so search.py can use them.
4  import joblib, numpy as np
5  from sentence_transformers import SentenceTransformer
6
7  model = SentenceTransformer("models/icai-embeddings")
8
9  CORPUS_FILES = [
10     ("embeddings_foundation.joblib", "corpus_foundation"),
11     ("embeddings_intermediate.joblib", "corpus_intermediate"),
12     ("embeddings_final.joblib", "corpus_final"),
13 ]
14
15 for joblib_path, corpus_attr in CORPUS_FILES:
16     print(f"Processing {joblib_path}...")
17
18     # Load existing joblib -- it contains {"chunks": [...], "embeddings":
19     np.ndarray}
20     data = joblib.load(joblib_path)

```

```

20     chunks = data["chunks"]    # adjust key name to match your actual structure
21
22     print(f"  Re-embedding {len(chunks)} chunks...")
23     new_embeddings = model.encode(
24         chunks,
25         batch_size=64,
26         show_progress_bar=True,
27         normalize_embeddings=True, # important for cosine similarity
28     )
29
30     # Save with same structure but new embeddings
31     data["embeddings"] = new_embeddings
32     joblib.dump(data, joblib_path)
33     print(f"  Saved {joblib_path} with shape {new_embeddings.shape}")
34
35 print("All .joblib files updated with fine-tuned embeddings.")

```

● 5.4 Update search.py to remove Ollama

```

1  # search.py -- replace the Ollama embedding call
2
3  from sentence_transformers import SentenceTransformer
4
5  # Load once at module level (not on every request)
6  _embed_model = SentenceTransformer("models/icai-embeddings")
7
8  def get_query_embedding(text: str) -> list[float]:
9      """
10     Replaces the Ollama HTTP call.
11     Runs in-process -- no sidecar needed, works in cloud deployment.
12     """
13     return _embed_model.encode(
14         text,
15         normalize_embeddings=True
16     ).tolist()
17
18 # Replace every call to ollama.embeddings() or requests.post(OLLAMA_URL)
19 # with get_query_embedding(text) in your existing search_rag() function.
20 # The rest of search.py (cosine similarity, CrossEncoder reranker) stays
21 # exactly the same.

```

How to test

```

1  # 1. Fine-tune (takes ~10-20 mins on CPU, ~3 mins with GPU):
2  python fine_tune_embeddings.py
3
4  # 2. Rebuild .joblib files:
5  python regenerate_joblib.py
6
7  # 3. Restart backend (no Ollama needed now):
8  uvicorn api:app --reload --host 127.0.0.1 --port 8000
9
10 # 4. Re-run RAGAS -- compare scores to Step 4 baseline:
11 python run_eval.py
12 # Context recall should improve. Document the improvement.

```

```

1 git add fine_tune_embeddings.py regenerate_joblib.py search.py models/
2 git commit -m "step-5: fine-tuned ICAI embeddings, removed Ollama dependency"

```

7. Step 6 — GraphRAG on Income Tax Act

Goal

Build a directed knowledge graph of IT Act cross-references and blend graph traversal with vector retrieval so multi-hop legal questions get complete answers.

Files touched

New: build_tax_graph.py New: graph_retrieve.py Edit: search.py

• 6.1 Install networkx

```
1 pip install networkx --break-system-packages
```

• 6.2 Create build_tax_graph.py

```
1 # build_tax_graph.py -- run once to build and save the graph
2 import re, joblib, networkx as nx
3
4 # Load your corpus chunks (same source as your .joblib embeddings)
5 # Each chunk should be a string of ICAI / IT Act text.
6 foundation = joblib.load("embeddings_foundation.joblib")
7 intermediate = joblib.load("embeddings_Intermediate.joblib")
8 final = joblib.load("embeddings_Final.joblib")
9
10 all_chunks = (
11     foundation["chunks"] +
12     intermediate["chunks"] +
13     final["chunks"]
14 )
15
16 # Pattern to detect section references in text
17 # Matches: Section 80C, Sec. 44AD, s.10(13A), Section 10(14)(i)
18 SEC_PATTERN = re.compile(
19     r"[Ss]ec(?:tion|\.)\s*([\d]+(?:\([A-Za-z0-9]+\)))*(?:\([ivxIVX]+\))*)"
20 )
21
22 G = nx.DiGraph()
23
24 for i, chunk in enumerate(all_chunks):
25     # Use chunk index as the "section ID" if no explicit section label
26     # Better: parse section header from chunk text
27     header_match = re.search(r"Section\s+([\d\w\(\)]+)", chunk[:100])
28     node_id = header_match.group(1) if header_match else f"chunk_{i}"
29
30     G.add_node(node_id, text=chunk[:500], chunk_index=i)
31
32     # Find all section references within this chunk
33     refs = SEC_PATTERN.findall(chunk)
34     for ref in set(refs):
35         if ref != node_id:
36             G.add_edge(node_id, ref, relation="references")
37
38 print(f"Graph: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")
39 joblib.dump(G, "tax_graph.joblib")
40 print("Saved to tax_graph.joblib")
```

● 6.3 Create graph_retrieve.py

```

1  # graph_retrieve.py
2  import joblib, networkx as nx
3  from typing import Optional
4
5  _graph: Optional[nx.DiGraph] = None
6
7  def _get_graph() -> nx.DiGraph:
8      global _graph
9      if _graph is None:
10         _graph = joblib.load("tax_graph.joblib")
11     return _graph
12
13  def graph_retrieve(seed_sections: list[str], hops: int = 2) -> list[dict]:
14      """
15      Given a list of section IDs found by vector retrieval,
16      traverse the cross-reference graph up to 'hops' steps
17      and return the text of all reachable nodes.
18
19      Args:
20         seed_sections: section IDs from vector retrieval (e.g. ["80C", "10"])
21         hops:          graph traversal depth (default 2)
22      Returns:
23         list of {"section": str, "text": str, "distance": int}
24      """
25      G = _get_graph()
26      results = []
27
28      for seed in seed_sections:
29         if seed not in G:
30             continue
31         # BFS from seed node
32         reachable = nx.single_source_shortest_path_length(G, seed,
33             cutoff=hops)
34         for node, dist in reachable.items():
35             node_data = G.nodes[node]
36             results.append({
37                 "section": node,
38                 "text": node_data.get("text", ""),
39                 "distance": dist,
40                 "source": "graph",
41             })
42
43     # Deduplicate by section
44     seen = set()
45     unique = []
46     for r in sorted(results, key=lambda x: x["distance"]):
47         if r["section"] not in seen:
48             seen.add(r["section"])
49             unique.append(r)
50
51     return unique
52
53  def extract_section_ids(vector_chunks: list[dict]) -> list[str]:
54      """Pull section IDs from the top vector-retrieved chunks."""
55      import re
56      ids = []
57      pattern = re.compile(r"Section\s+([\d\w\(\)]+)")
58      for chunk in vector_chunks:
59         matches = pattern.findall(chunk.get("text", ""))
60         ids.extend(matches)
61     return list(set(ids))

```

• 6.4 Blend graph results into search.py

```

1  # search.py -- in your search_rag() function, after vector retrieval:
2  from graph_retrieve import graph_retrieve, extract_section_ids
3
4  def search_rag(query: str, top_k: int = 5) -> list[dict]:
5      # --- existing vector retrieval ---
6      query_emb = get_query_embedding(query)
7      vector_chunks = cosine_retrieve(query_emb, top_k=top_k * 2)
8      # --- existing CrossEncoder reranking ---
9      reranked = cross_encode_rerank(query, vector_chunks, top_k=top_k)
10
11     # --- NEW: graph expansion ---
12     seed_ids = extract_section_ids(reranked)
13     graph_chunks = graph_retrieve(seed_ids, hops=2)
14
15     # Merge: vector results first, then graph neighbours not already included
16     seen_texts = {c["text"][:100] for c in reranked}
17     merged = list(reranked)
18     for g in graph_chunks:
19         if g["text"][:100] not in seen_texts and g["distance"] > 0:
20             merged.append(g)
21             seen_texts.add(g["text"][:100])
22
23     # Re-rank the merged set one more time
24     final = cross_encode_rerank(query, merged, top_k=top_k)
25     return final

```

How to test

```

1  # 1. Build the graph:
2  python build_tax_graph.py
3  # Should print: "Graph: NNN nodes, MMM edges"
4
5  # 2. Restart backend and ask a multi-hop question in the UI:
6  # "Can I claim HRA exemption and home loan deduction simultaneously?"
7  # The answer should now reference Section 10(13A), Section 24,
8  # and Section 80C together.
9
10 # 3. Re-run RAGAS on the multi-hop subset of your golden dataset.

```

```

1  git add build_tax_graph.py graph_retrieve.py search.py
2  git commit -m "step-6: GraphRAG hybrid retrieval on IT Act cross-references"

```

8. Step 7 — Hallucination Detection and Source Badges

Goal

After every generated answer, run an NLI check to determine whether each claim is entailed by the retrieved chunks. Display a verified badge or an “inferred” warning in the chat UI.

Files touched

New: hallucination_guard.py Edit: api.py Edit: src/components/ChatMessage.tsx

• 7.1 Create `hallucination_guard.py`

```

1  # hallucination_guard.py
2  from sentence_transformers import CrossEncoder
3
4  # ~90 MB model, CPU-friendly, no GPU required
5  _nli_model: CrossEncoder = None
6
7  def _get_nli() -> CrossEncoder:
8      global _nli_model
9      if _nli_model is None:
10         _nli_model = CrossEncoder("cross-encoder/nli-MiniLM2-L6-H768")
11     return _nli_model
12
13 def check_grounding(
14     answer: str,
15     chunks: list[str],
16     entailment_threshold: float = 0.65
17 ) -> dict:
18     """
19     Check whether the answer is grounded in the retrieved chunks.
20
21     Args:
22         answer: the generated answer text
23         chunks: list of retrieved chunk texts
24         entailment_threshold: minimum entailment score to call it verified
25
26     Returns:
27         {
28             "verified": bool,
29             "score": float, # highest entailment score
30             "source": str | None, # excerpt of best supporting chunk
31             "badge_label": str, # display text for UI
32             "badge_color": str, # "green" | "amber"
33         }
34     """
35     if not chunks:
36         return {
37             "verified": False, "score": 0.0, "source": None,
38             "badge_label": "No sources retrieved", "badge_color": "amber"
39         }
40
41     nli = _get_nli()
42     pairs = [(chunk[:512], answer[:256]) for chunk in chunks]
43
44     # scores shape: (N, 3) -- [contradiction, neutral, entailment]
45     scores = nli.predict(pairs, apply_softmax=True)
46     entail_col = 2 # entailment is column index 2 for this model
47     entail_scores = scores[:, entail_col]
48     best_idx = int(entail_scores.argmax())
49     best_score = float(entail_scores[best_idx])
50
51     verified = best_score >= entailment_threshold
52     return {
53         "verified": verified,
54         "score": round(best_score, 3),
55         "source": chunks[best_idx][:120] + "..." if verified else None,
56         "badge_label": "ICAI-verified" if verified else "Inferred -- verify",
57         "badge_color": "green" if verified else "amber",
58     }

```

• 7.2 Call it in api.py

```

1 # api.py -- in /ask endpoint, after getting the answer:
2 from hallucination_guard import check_grounding
3
4 # After: answer = process_query(...)
5 chunk_texts = [c.get("text", "") for c in top_chunks]
6 grounding = check_grounding(answer, chunk_texts)
7
8 return {
9     "answer":          answer,
10    "summary":         new_summary,
11    "retrieval_chunks": chunk_texts,
12    "retrieval":       top_chunks,
13    "grounding":       grounding,      # NEW -- includes badge info
14 }

```

• 7.3 Render the badge in ChatMessage.tsx

```

1 // src/components/ChatMessage.tsx
2 // Add badge rendering below the answer text:
3
4 interface Grounding {
5     verified:    boolean;
6     score:       number;
7     source:      string | null;
8     badge_label: string;
9     badge_color: "green" | "amber";
10 }
11
12 interface Message {
13     role:        "user" | "assistant";
14     content:     string;
15     grounding?: Grounding; // NEW
16 }
17
18 // In the JSX for assistant messages, add after the answer text:
19 {message.grounding && (
20     <div className={'mt-2 flex items-center gap-2 text-xs font-medium
21         ${message.grounding.badge_color === "green"
22             ? "text-green-700" : "text-amber-700"}'}>
23         <span className={'w-2 h-2 rounded-full
24             ${message.grounding.badge_color === "green"
25                 ? "bg-green-500" : "bg-amber-500"}'} />
26         {message.grounding.badge_label}
27         {message.grounding.source && (
28             <span className="font-normal text-gray-500 ml-1">
29                 ({message.grounding.source})
30             </span>
31         )}
32     </div>
33 )}

```

How to test

Ask: “What is the GST registration threshold for a service provider?” The answer should show a green ICAI-verified badge. Ask a trick question on a topic not in ICAI material — the badge should show amber.

```

1 git add hallucination_guard.py api.py src/components/ChatMessage.tsx
2 git commit -m "step-7: NLI hallucination detection with source badges"

```

9. Step 8 — Agentic ITR Filing Assistant

Goal

Build a 4-step guided agent that classifies the correct ITR form, asks for missing information, pre-fills available fields, and exports a summary checklist.

Files touched

New: `itr_agent.py` Edit: `api.py` New: `src/components/ITRWizard.tsx`

• 8.1 Create `itr_agent.py`

```

1  # itr_agent.py
2  import google.generativeai as genai
3
4  CLASSIFY_PROMPT = """
5  You are a CA assistant. Based on the following financial profile,
6  determine which ITR form the taxpayer should file.
7
8  Rules:
9  - ITR-1 (Sahaj): Salaried individual, income <= 50L, one house property, no
    business income.
10 - ITR-2: Individual/HUF, no business income, income > 50L or capital gains.
11 - ITR-3: Individual/HUF with business income (not presumptive).
12 - ITR-4 (Sugam): Presumptive income under Section 44AD/44ADA/44AE.
13
14 Financial profile:
15 {profile}
16
17 Respond with ONLY a JSON object:
18 [{"form": "ITR-1", "reason": "short reason", "confidence": 0.95}]
19 """
20
21 GAP_PROMPT = """
22 Based on the ITR form {form} and the available data below,
23 list what information is still missing to complete the filing.
24
25 Available data:
26 {available}
27
28 Respond with ONLY a JSON array of missing items:
29 [{"field": "Form 16", "reason": "Required to verify TDS from employer"}]]
30 """
31
32 class ITRAgent:
33     STEPS = ["classify", "gap_fill", "pre_fill", "export"]
34
35     def __init__(self, doc_data: dict):
36         self.doc_data = doc_data
37         self.state = {
38             "step": 0,
39             "itr_form": None,
40             "reason": None,
41             "missing_fields": [],
42             "prefilled": {},
43             "extra_answers": {},
44         }
45         self.model = genai.GenerativeModel("gemini-1.5-flash")

```

```

46
47     def _classify(self) -> dict:
48         profile = str(self.doc_data)
49         resp = self.model.generate_content(
50             CLASSIFY_PROMPT.format(profile=profile)
51         )
52         import json, re
53         raw = re.sub(r"```json|```", "", resp.text).strip()
54         data = json.loads(raw)
55         self.state["itr_form"] = data["form"]
56         self.state["reason"] = data["reason"]
57         return {
58             "message": (
59                 f"Based on your financial profile, you should file "
60                 f"```{data['form']}```. \n\n{data['reason']}"
61             ),
62             "step_complete": "classify",
63         }
64
65     def _gap_fill(self) -> dict:
66         import json, re
67         available = str({k: v for k, v in self.doc_data.items()
68                         if v and str(v) != "null"})
69         resp = self.model.generate_content(
70             GAP_PROMPT.format(form=self.state["itr_form"],
71                               available=available)
72         )
73         raw = re.sub(r"```json|```", "", resp.text).strip()
74         missing = json.loads(raw)
75         self.state["missing_fields"] = missing
76         if not missing:
77             return {"message": "All required information is available.
78                     Proceeding to pre-fill.",
79                     "step_complete": "gap_fill"}
80         items = "\n".join(f"- **{m['field']}**: {m['reason']}" for m in
81                             missing)
82         return {
83             "message": f"The following information is still
84                         needed:\n{items}\n\nPlease provide these or upload the
85                         relevant documents.",
86             "step_complete": "gap_fill",
87             "missing_fields": missing,
88         }
89
90     def _pre_fill(self) -> dict:
91         """Map extracted doc_data fields to ITR field names."""
92         mapping = {
93             "Gross Salary / Total Income":
94                 self.doc_data.get("estimated_annual_income"),
95             "TDS Deducted (Part B)": self.doc_data.get("tds_deducted"),
96             "Opening Balance":
97                 self.doc_data.get("opening_balance"),
98             "Closing Balance":
99                 self.doc_data.get("closing_balance"),
100             "Name of Taxpayer": self.doc_data.get("person_entity"),
101         }
102         self.state["prefilled"] = {k: v for k, v in mapping.items() if v}
103         filled_lines = "\n".join(
104             f"- **{k}**: {v}" for k, v in self.state["prefilled"].items()
105         )
106         missing_count = len(self.state["missing_fields"])
107         return {
108             "message": (

```

```

101         f"Pre-filled {len(self.state['prefilled'])} fields "
102         f"from your uploaded documents:\n\n{filled_lines}\n\n"
103         f"{missing_count} field(s) still need manual entry."
104     ),
105     "step_complete": "pre_fill",
106     "prefilled": self.state["prefilled"],
107 }
108
109 def _export(self) -> dict:
110     missing_lines = "\n".join(
111         f"- [ ] {m['field']} -- {m['reason']}"
112         for m in self.state["missing_fields"]
113     )
114     filled_lines = "\n".join(
115         f"- [x] {k}: {v}" for k, v in self.state["prefilled"].items()
116     )
117     checklist = (
118         f"# {self.state['itr_form']} Filing Checklist\n\n"
119         f"## Auto-filled from documents\n{filled_lines}\n\n"
120         f"## Needs manual input\n{missing_lines}\n"
121     )
122     self.state["checklist"] = checklist
123     return {
124         "message": "Your ITR filing checklist is ready. Download it below.",
125         "step_complete": "export",
126         "checklist": checklist,
127     }
128
129 def run_next_step(self) -> dict:
130     """Advance to the next step and return the result."""
131     step_idx = self.state["step"]
132     if step_idx >= len(self.STEPS):
133         return {"message": "Filing checklist is complete.", "done": True}
134     step_name = self.STEPS[step_idx]
135     self.state["step"] += 1
136     handler = {
137         "classify": self._classify,
138         "gap_fill": self._gap_fill,
139         "pre_fill": self._pre_fill,
140         "export": self._export,
141     }[step_name]
142     return handler()

```

• 8.2 Add /itr endpoint in api.py

```

1  # api.py
2  from itr_agent import ITRAgent
3
4  class ITRRequest(BaseModel):
5      doc_data: dict = {}
6      agent_state: dict | None = None  # passed back from frontend each step
7
8  @app.post("/itr")
9  async def itr_step(req: ITRRequest):
10     agent = ITRAgent(doc_data=req.doc_data)
11     # Restore state if not first step
12     if req.agent_state:
13         agent.state = req.agent_state
14     result = agent.run_next_step()
15     result["state"] = agent.state  # send state back to frontend
16     return result

```

● 8.3 Create ITRWizard.tsx

```

1 // src/components/ITRWizard.tsx
2 import { useState } from "react";
3 import { useChat } from "../contexts/ChatContext";
4
5 const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8000";
6 const STEP_LABELS = ["Classify Form", "Identify Gaps", "Pre-fill Fields",
7   "Export Checklist"];
8
9 export default function ITRWizard() {
10   const { activeConversation } = useChat();
11   const [agentState, setAgentState] = useState(null);
12   const [stepIndex, setStepIndex] = useState(0);
13   const [messages, setMessages] = useState<string[]>([]);
14   const [loading, setLoading] = useState(false);
15   const [checklist, setChecklist] = useState<string | null>(null);
16
17   const docData = activeConversation?.docData?.[0] ?? {};
18
19   const runNextStep = async () => {
20     setLoading(true);
21     const res = await fetch(`${API_URL}/itr`, {
22       method: "POST",
23       headers: { "Content-Type": "application/json" },
24       body: JSON.stringify({ doc_data: docData, agent_state: agentState }),
25     });
26     const data = await res.json();
27     setLoading(false);
28     setAgentState(data.state);
29     setMessages((prev) => [...prev, data.message]);
30     if (data.checklist) setChecklist(data.checklist);
31     if (!data.done) setStepIndex((i) => i + 1);
32   };
33
34   return (
35     <div className="max-w-2xl mx-auto p-6 space-y-6">
36       <h2 className="text-xl font-semibold text-gray-800">ITR Filing
37         Assistant</h2>
38
39       { /* Progress bar */ }
40       <div className="flex gap-2">
41         {STEP_LABELS.map((label, i) => (
42           <div key={i} className={`flex-1 h-1.5 rounded-full
43             ${i < stepIndex ? "bg-purple-500" : i === stepIndex ?
44               "bg-purple-300" : "bg-gray-200"}`} />
45         ))}
46       </div>
47       <p className="text-sm text-gray-500">
48         {stepIndex < STEP_LABELS.length ? `Step ${stepIndex + 1}:
49           ${STEP_LABELS[stepIndex]}` : "Complete"}
50       </p>
51
52       { /* Messages */ }
53       <div className="space-y-3">
54         {messages.map((msg, i) => (
55           <div key={i} className="bg-gray-50 border border-gray-200 rounded-lg
56             p-4 text-sm text-gray-800 whitespace-pre-wrap">
57             {msg}
58           </div>
59         ))}
60       </div>
61
62       { /* Checklist download */ }

```

```

58     {checklist && (
59         <a
60             href={`data:text/plain;charset=utf-8,${encodeURIComponent(checklist)}`}
61             download="itr_checklist.md"
62             className="inline-block bg-teal-600 text-white px-4 py-2 rounded-lg
63                 text-sm">
64             Download Checklist
65         </a>
66     )}
67
68     {/* Next step button */}
69     {stepIndex < STEP_LABELS.length && (
70         <button onClick={runNextStep} disabled={loading}
71             className="bg-purple-600 text-white px-5 py-2 rounded-lg text-sm
72                 disabled:opacity-50">
73             {loading ? "Processing..." : stepIndex === 0 ? "Start Filing
74                 Assistant" : "Next Step"}
75         </button>
76     )}
77 </div>
78 );
79 }

```

How to test

Upload a salary-based bank statement. Navigate to the ITR Wizard (add a route `/itr` in `App.tsx` pointing to `ITRWizard`). Click through all 4 steps. On the final step, download the checklist markdown file.

```

1 git add itr_agent.py api.py src/components/ITRWizard.tsx
2 git commit -m "step-8: agentic 4-step ITR filing assistant"

```

10. Step 9 — Compliance Calendar and Deadline Alerts

Goal

Generate personalised GST and income tax deadlines from the user's financial profile and surface them as browser push notifications.

Files touched

New: `compliance_calendar.py` Edit: `api.py` New: `src/components/CalendarView.tsx`
 New: `public/sw.js` (service worker)

• 9.1 Create `compliance_calendar.py`

```

1 # compliance_calendar.py
2 from datetime import date, timedelta
3 from calendar import monthrange
4
5 def _next_occurrence(day: int, month_offset: int = 0) -> date:
6     """Return the next date with this day-of-month."""
7     today = date.today()
8     year = today.year
9     month = today.month + month_offset

```

```

10     if month > 12:
11         year += 1
12         month -= 12
13     last_day = monthrange(year, month)[1]
14     return date(year, month, min(day, last_day))
15
16 def generate_calendar(doc_data: dict) -> list[dict]:
17     """
18     Generate compliance deadlines personalised to the taxpayer's profile.
19
20     Reads from doc_data:
21     - gst_registered (bool or "yes"/"no")
22     - gst_filing_frequency ("monthly" | "quarterly")
23     - estimated_annual_income (str or float)
24     """
25     deadlines = []
26     today = date.today()
27     year = today.year
28
29     gst_reg = str(doc_data.get("gst_registered", "")).lower() in ("yes",
30         "true", "1")
31     freq = str(doc_data.get("gst_filing_frequency", "monthly")).lower()
32     income = float(str(doc_data.get("estimated_annual_income", "0")
33         ).replace(",", "").replace("Rs.", "").strip() or 0)
34
35     # --- GST deadlines ---
36     if gst_reg:
37         if freq == "monthly":
38             for m in range(1, 13):
39                 deadlines.append({
40                     "name": "GSTR-1",
41                     "due_date": date(year, m, 11).isoformat(),
42                     "type": "GST",
43                     "note": "Monthly GSTR-1 filing deadline",
44                 })
45             deadlines.append({
46                 "name": "GSTR-3B",
47                 "due_date": date(year, m, 20).isoformat(),
48                 "type": "GST",
49                 "note": "Monthly GSTR-3B payment and filing",
50             })
51         else: # quarterly
52             for q_month in [4, 7, 10, 1]:
53                 q_year = year if q_month != 1 else year + 1
54                 deadlines.append({
55                     "name": "GSTR-1 (Quarterly)",
56                     "due_date": date(q_year, q_month, 13).isoformat(),
57                     "type": "GST",
58                     "note": "Quarterly GSTR-1 deadline",
59                 })
60
61     # --- Advance Tax (if income > 2.5L) ---
62     if income > 250000:
63         for q_date, pct in [
64             (date(year, 6, 15), 15),
65             (date(year, 9, 15), 45),
66             (date(year, 12, 15), 75),
67             (date(year + 1, 3, 15), 100),
68         ]:
69             deadlines.append({
70                 "name": f"Advance Tax

```

```

71         "type":      "Income Tax",
72         "note":      f"Cumulative {pct}% of advance tax due",
73     })
74
75     # --- ITR filing deadline ---
76     deadlines.append({
77         "name":      "ITR Filing",
78         "due_date":  date(year, 7, 31).isoformat(),
79         "type":      "Income Tax",
80         "note":      "Individual ITR filing deadline (non-audit)",
81     })
82
83     # Sort by due date and filter to upcoming only
84     deadlines.sort(key=lambda x: x["due_date"])
85     upcoming = [d for d in deadlines if d["due_date"] >= today.isoformat()]
86     return upcoming[:20] # return next 20 deadlines

```

● 9.2 Add /calendar endpoint

```

1  # api.py
2  from compliance_calendar import generate_calendar
3
4  class CalendarRequest(BaseModel):
5      doc_data: dict = {}
6
7  @app.post("/calendar")
8  async def calendar(req: CalendarRequest):
9      deadlines = generate_calendar(req.doc_data)
10     return {"deadlines": deadlines}

```

● 9.3 Create CalendarView.tsx with push notifications

```

1  // src/components/CalendarView.tsx
2  import { useEffect, useState } from "react";
3  import { useChat } from "../contexts/ChatContext";
4
5  const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8000";
6
7  interface Deadline {
8      name: string; due_date: string; type: string; note: string;
9  }
10
11  function daysUntil(dateStr: string): number {
12      return Math.ceil((new Date(dateStr).getTime() - Date.now()) / 86400000);
13  }
14
15  export default function CalendarView() {
16      const { activeConversation } = useChat();
17      const [deadlines, setDeadlines] = useState<Deadline[]>([]);
18
19      useEffect(() => {
20          const docData = activeConversation?.docData?.[0] ?? {};
21          fetch(`${API_URL}/calendar`, {
22              method: "POST",
23              headers: { "Content-Type": "application/json" },
24              body: JSON.stringify({ doc_data: docData }),
25          })
26              .then((r) => r.json())
27              .then((d) => setDeadlines(d.deadlines ?? []))
28              .catch(console.error);
29      }, [activeConversation]);

```

```

30
31 const requestNotifications = async () => {
32   if (!("Notification" in window)) return;
33   const perm = await Notification.requestPermission();
34   if (perm !== "granted") return;
35   // Schedule notifications for deadlines within 3 days
36   deadlines.forEach((d) => {
37     const days = daysUntil(d.due_date);
38     if (days >= 0 && days <= 3) {
39       new Notification('CA Agent: ${d.name} due in ${days} day(s)', {
40         body: d.note,
41         icon: "/favicon.ico",
42       });
43     }
44   });
45 };
46
47 return (
48   <div className="max-w-2xl mx-auto p-6">
49     <div className="flex items-center justify-between mb-4">
50       <h2 className="text-xl font-semibold text-gray-800">Compliance
51         Calendar</h2>
52       <button onClick={requestNotifications}
53         className="text-sm bg-teal-600 text-white px-3 py-1.5 rounded-lg">
54         Enable Reminders
55       </button>
56     </div>
57
58     <div className="space-y-3">
59       {deadlines.map((d, i) => {
60         const days = daysUntil(d.due_date);
61         const urgent = days <= 7;
62         return (
63           <div key={i} className={`p-4 rounded-xl border
64             ${urgent ? "border-red-200 bg-red-50" : "border-gray-200
65               bg-white"}>
66             <div className="flex items-center justify-between">
67               <div>
68                 <p className="font-medium text-gray-900 text-sm">{d.name}</p>
69                 <p className="text-xs text-gray-500 mt-0.5">{d.note}</p>
70               </div>
71               <div className="text-right">
72                 <p className="text-sm font-semibold text-gray-800">
73                   {new Date(d.due_date).toLocaleDateString("en-IN",
74                     { day: "numeric", month: "short", year: "numeric" })}
75                 </p>
76                 <p className={`text-xs mt-0.5 ${urgent ? "text-red-600
77                   font-medium" : "text-gray-400"}>
78                   {days === 0 ? "Due today" : days < 0 ? "Overdue" : `${days}
79                     days left`}
80                 </p>
81               </div>
82             </div>
83           </div>
84         );
85       })}
86     </div>
87   </div>
88 );
89 }

```

How to test

Upload a statement for a GST-registered entity. Navigate to `/calendar`. You should see GSTR-1 and GSTR-3B deadlines listed for each month, advance tax quarters, and the ITR deadline. Click “Enable Reminders” and check that browser notification permission is requested.

```
1 git add compliance_calendar.py api.py src/components/CalendarView.tsx
2 git commit -m "step-9: compliance calendar with browser push notifications"
```

11. Step 10 — Voice Interface

Goal

Add speech-to-text input and text-to-speech output using the browser’s built-in Web Speech API. Zero backend changes. Zero API cost.

Files touched

New:	<code>src/hooks/useSpeech.ts</code>	Edit:	<code>src/components/ChatInput.tsx</code>	Edit:
	<code>src/components/ChatMessage.tsx</code>			

• 10.1 Create useSpeech.ts hook

```
1 // src/hooks/useSpeech.ts
2 import { useState, useCallback, useRef } from "react";
3
4 export function useSpeechInput(onResult: (text: string) => void) {
5   const [listening, setListening] = useState(false);
6   const recognitionRef = useRef<any>(null);
7
8   const startListening = useCallback(() => {
9     const SpeechRecognition =
10       (window as any).SpeechRecognition ||
11       (window as any).webkitSpeechRecognition;
12     if (!SpeechRecognition) {
13       alert("Speech recognition is not supported in this browser. Use Chrome.");
14       return;
15     }
16     const recognition = new SpeechRecognition();
17     recognition.lang = "en-IN"; // Indian English
18     recognition.interimResults = false;
19     recognition.maxAlternatives = 1;
20
21     recognition.onresult = (e: any) => {
22       const transcript = e.results[0][0].transcript;
23       onResult(transcript);
24       setListening(false);
25     };
26     recognition.onerror = () => setListening(false);
27     recognition.onend = () => setListening(false);
28
29     recognitionRef.current = recognition;
30     recognition.start();
31     setListening(true);
32   }, [onResult]);
33 }
```

```

34   const stopListening = useCallback(() => {
35     recognitionRef.current?.stop();
36     setListening(false);
37   }, []);
38
39   return { listening, startListening, stopListening };
40 }
41
42 export function speakText(text: string, lang: string = "en-IN") {
43   if (!("speechSynthesis" in window)) return;
44   window.speechSynthesis.cancel(); // stop any ongoing speech
45   const utterance = new SpeechSynthesisUtterance(text);
46   utterance.lang = lang;
47   utterance.rate = 0.95;
48   window.speechSynthesis.speak(utterance);
49 }

```

● 10.2 Add mic button to ChatInput.tsx

```

1  // src/components/ChatInput.tsx -- add mic button
2  import { useSpeechInput } from "../hooks/useSpeech";
3
4  // Inside your ChatInput component:
5  const { listening, startListening, stopListening } = useSpeechInput(
6    (text) => setInput((prev) => prev + (prev ? " " : "") + text)
7  );
8
9  // In JSX, add mic button next to send button:
10 <button
11   type="button"
12   onClick={listening ? stopListening : startListening}
13   className={`p-2 rounded-lg border ${
14     listening
15       ? "bg-red-100 border-red-300 text-red-600 animate-pulse"
16       : "bg-gray-100 border-gray-200 text-gray-500 hover:bg-gray-200"
17   }`}
18   title={listening ? "Stop listening" : "Speak your question"}
19 >
20   { /* Microphone SVG icon */}
21   <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
22     stroke="currentColor" strokeWidth="2" strokeLinecap="round"
23     strokeLinejoin="round">
24     <rect x="9" y="2" width="6" height="12" rx="3"/>
25     <path d="M19 10a7 7 0 0 1-14 0M12 19v3M8 22h8"/>
26   </svg>
27 </button>

```

● 10.3 Add read-aloud button to ChatMessage.tsx

```

1  // src/components/ChatMessage.tsx -- add speaker button to assistant messages
2  import { speakText } from "../hooks/useSpeech";
3
4  // Inside the assistant message JSX:
5  {message.role === "assistant" && (
6    <button
7      onClick={() => speakText(message.content)}
8      className="mt-2 text-xs text-gray-400 hover:text-gray-600 flex items-center
9        gap-1"
10    >
11      <svg width="14" height="14" viewBox="0 0 24 24" fill="none"
12        stroke="currentColor" strokeWidth="2" strokeLinecap="round">

```

```

12     <polygon points="11 5 6 9 2 9 2 15 6 15 11 19 11 5"/>
13     <path d="M19.07 4.93a10 10 0 0 1 0 14.14M15.54 8.46a5 5 0 0 1 0 7.07"/>
14     </svg>
15     Read aloud
16 </button>
17 }}

```

How to test

Open the app in Chrome (required for Web Speech API). Click the mic button and say: “*What is the due date for GSTR-3B?*” The text should appear in the input box. Send it and click “Read aloud” on the response.

```

1 git add src/hooks/useSpeech.ts src/components/ChatInput.tsx
  src/components/ChatMessage.tsx
2 git commit -m "step-10: voice input and text-to-speech via Web Speech API"

```

12. Step 11 — Containerise the Backend

Goal

Package the entire backend — Python dependencies, fine-tuned model, and `.joblib` embedding files — into a single Docker image so it deploys identically anywhere.

Files touched

New: `Dockerfile` New: `docker-compose.yml` New: `.dockerignore`

● 11.1 Create `requirements.txt`

```

1 # Generate from your current environment:
2 pip freeze > requirements.txt
3
4 # Then manually clean it to only what your project needs.
5 # Minimum required lines (add others your api.py imports):

```

```

1 # requirements.txt -- paste this as a starting point
2 fastapi==0.110.0
3 uvicorn[standard]==0.29.0
4 python-multipart==0.0.9
5 python-dotenv==1.0.1
6 pydantic==2.6.4
7 google-generativeai==0.5.4
8 sentence-transformers==2.7.0
9 scikit-learn==1.4.2
10 numpy==1.26.4
11 joblib==1.4.0
12 networkx==3.3
13 torch==2.2.2
14 ragas==0.1.7
15 datasets==2.18.0

```

● 11.2 Create `Dockerfile`

```

1  # Dockerfile
2  FROM python:3.11-slim
3
4  # Install system dependencies for sentence-transformers
5  RUN apt-get update && apt-get install -y \
6      gcc g++ libgomp1 \
7      && rm -rf /var/lib/apt/lists/*
8
9  WORKDIR /app
10
11 # Install Python dependencies first (layer cache)
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 # Copy embedding model checkpoint
16 COPY models/ models/
17
18 # Copy precomputed corpus embeddings
19 COPY embeddings_foundation.joblib .
20 COPY embeddings_Intermediate.joblib .
21 COPY embeddings_Final.joblib .
22 COPY tax_graph.joblib .
23
24 # Copy all Python source files
25 COPY *.py .
26
27 # Expose FastAPI port
28 EXPOSE 8000
29
30 # Production command (no --reload in production)
31 CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]

```

● 11.3 Create docker-compose.yml

```

1  # docker-compose.yml -- for local testing before cloud deployment
2  version: "3.9"
3  services:
4      backend:
5          build: .
6          ports:
7              - "8000:8000"
8          env_file:
9              - .env
10         volumes:
11             - ./temp:/app/temp      # uploaded files directory
12
13     frontend:
14         image: node:20-alpine
15         working_dir: /app
16         volumes:
17             - ./app
18         ports:
19             - "5173:5173"
20         command: sh -c "cd frontend && npm install && npm run dev -- --host"
21         environment:
22             - VITE_API_URL=http://localhost:8000
23         depends_on:
24             - backend

```

● 11.4 Create .dockerignore

```

1 # .dockerignore
2 .env
3 .git
4 __pycache__
5 *.pyc
6 node_modules
7 temp/
8 *.log
9 .venv
10 venv/
11 frontend/node_modules

```

How to test

```

1 # Build and run the container locally:
2 docker build -t ca-agent-backend .
3 docker run -p 8000:8000 --env-file .env ca-agent-backend
4
5 # In another terminal, test the API:
6 curl -X POST http://localhost:8000/ask \
7   -H "Content-Type: application/json" \
8   -d '{"question":"What is Section 44AD?","history":[]}'
9 # Should return a valid JSON response with an answer field.

```

```

1 git add Dockerfile docker-compose.yml .dockerignore requirements.txt
2 git commit -m "step-11: Dockerfile and docker-compose for backend"

```

13. Step 12 — Cloud Deployment

Deployment stack (all free tier)

Backend: Render.com (Docker deploy) or Railway.app

Frontend: Vercel (auto-detects Vite/React)

Fallback for demo day: Cloudflare Tunnel (no account needed)

• 12.1 Deploy backend to Render.com

1. Push your repo to GitHub.
2. Go to `render.com` → New → **Web Service**.
3. Connect your GitHub repository.
4. Choose **Docker** as the runtime (Render auto-detects your `Dockerfile`).
5. Set the following environment variables in the Render dashboard:

```

1 # Render environment variables (set via dashboard, NOT in .env file)
2 GEMINI_API_KEY=your_gemini_api_key_here
3 ENABLE_HYDE=false
4 ENABLE_METRICS=false

```

Handling large .joblib files

If your `.joblib` files exceed 100 MB combined, Git will refuse them. Use one of these options:

Option A – Git LFS:

```
1 git lfs install
2 git lfs track "*.joblib"
3 git add .gitattributes
4 git add *.joblib
5 git commit -m "add embeddings via git lfs"
```

Option B – Download at container startup:

```
1 # startup.py -- add to Dockerfile CMD or as entrypoint
2 from huggingface_hub import hf_hub_download
3 import os
4
5 FILES = ["embeddings_foundation.joblib",
6         "embeddings_intermediate.joblib",
7         "embeddings_final.joblib"]
8 REPO = "your-hf-username/ca-agent-embeddings"
9
10 for f in FILES:
11     if not os.path.exists(f):
12         print(f"Downloading {f} from Hugging Face Hub...")
13         hf_hub_download(repo_id=REPO, filename=f, local_dir=".")
```

● 12.2 Deploy frontend to Vercel

```
1 # Install Vercel CLI
2 npm install -g vercel
3
4 # From your project root (or frontend/ directory):
5 cd frontend/ # if your React app is in a subdirectory
6 vercel
7
8 # Answer the prompts:
9 # - Link to existing project? No
10 # - Project name: ca-agent
11 # - Build command: npm run build
12 # - Output directory: dist
13
14 # Set the backend URL as an environment variable:
15 vercel env add VITE_API_URL
16 # Paste your Render backend URL: https://ca-agent-xxxx.onrender.com
17
18 # Deploy to production:
19 vercel --prod
```

● 12.3 Fix CORS for production

```
1 # api.py -- tighten CORS for production
2 import os
3 from fastapi.middleware.cors import CORSMiddleware
4
5 FRONTEND_URL = os.getenv("FRONTEND_URL", "http://localhost:5173")
6
7 app.add_middleware(
8     CORSMiddleware,
9     allow_origins=[FRONTEND_URL, "http://localhost:5173"],
```

```

10     allow_credentials=True,
11     allow_methods=["*"],
12     allow_headers=["*"],
13 )
14 # Add FRONTEND_URL=https://ca-agent.vercel.app to Render environment variables

```

● 12.4 Demo-day fallback: Cloudflare Tunnel

If Render is slow to wake up (free tier spins down after 15 min), use this for your viva / demo:

```

1 # Install cloudflared (one time):
2 # Windows: winget install Cloudflare.cloudflared
3 # Mac:      brew install cloudflared
4 # Linux:    see
5             https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/
6
7 # Run your backend locally:
8 uvicorn api:app --host 127.0.0.1 --port 8000
9
10 # In another terminal, create a public tunnel:
11 cloudflared tunnel --url http://localhost:8000
12 # Prints something like: https://abc-xyz.trycloudflare.com
13 # Share this URL with your examination panel.
14 # No account or login required.

```

How to test

```

1 # After both are deployed, test the full stack end-to-end:
2 # 1. Open your Vercel URL in a browser.
3 # 2. Upload a bank statement PDF.
4 # 3. Ask: "What is my estimated advance tax liability?"
5 # 4. Verify the answer shows an ICAI-verified badge.
6 # 5. Open /dashboard and check charts are populated.
7 # 6. Open /itr and run all 4 wizard steps.
8 # 7. Open /calendar and verify upcoming GST deadlines appear.
9 # 8. Click the mic button and speak a question.
10 # If all 8 work, the project is complete.

```

```

1 git add api.py
2 git commit -m "step-12: CORS update for production deployment"
3 git tag v1.0-fyp
4 git push origin fyp-extensions --tags

```

14. Final Checklist

Done	Step	Key file(s)	How to verify
☐	1. Rolling memory	<code>memory.py</code> , <code>api.py</code>	7-message session, context retained
☐	2. Multi-doc analysis	<code>ca_agent.py</code> , <code>Dashboard.tsx</code>	Dashboard charts populated
☐	3. Anomaly detection	<code>anomaly.py</code>	Flagged txns in upload response
☐	4. RAGAS harness	<code>run_eval.py</code>	Baseline scores printed
☐	5. Fine-tuned embeddings	<code>fine_tune_embeddings.py</code> , <code>search.py</code>	RAGAS improves vs baseline
☐	6. GraphRAG	<code>build_tax_graph.py</code> , <code>search.py</code>	Multi-hop answers include 3+ sections
☐	7. Hallucination badge	<code>hallucination_guard.py</code>	Green/amber badge visible in UI
☐	8. ITR wizard	<code>itr_agent.py</code> , <code>ITRWizard.tsx</code>	Checklist downloads on step 4
☐	9. Compliance calendar	<code>compliance_calendar.py</code>	Upcoming deadlines listed
☐	10. Voice interface	<code>useSpeech.ts</code>	Mic transcribes, read-aloud works
☐	11. Dockerfile	<code>Dockerfile</code>	<code>docker run</code> starts API
☐	12. Cloud deployment	Render + Vercel	Public URLs respond correctly